

嵌入式系统设计与开发个人总结报告

樊嘉乐

学号：3124001597

2026 年 7 月

目录

1	项目概述	1
1.1	项目背景	1
1.2	核心功能	1
1.3	项目意义	1
2	设计目标与预期成果	1
2.1	设计目标	1
2.2	预期成果	1
3	系统硬件设计	2
3.1	系统总体架构	2
3.2	核心器件选型	2
3.3	硬件引脚分配	2
3.4	电源管理设计	3
4	系统软件设计	3
4.1	软件整体架构	3
4.2	FreeRTOS 任务设计	3
4.3	系统主流程	4
4.4	UI 状态机设计	4
4.5	计步算法实现	4
4.5.1	算法原理	4
4.5.2	运动状态识别	5
4.6	OLED 显示驱动	5
4.7	蓝牙通信模块设计	5
4.7.1	硬件配置	5
4.7.2	数据协议设计	5
4.8	资源占用分析	6
5	PC 端上位机开发	6
5.1	开发环境与工具	6
5.2	软件架构	6
5.3	核心功能实现	6
5.3.1	BLE 连接管理	6
5.3.2	数据接收与解析	7
5.3.3	实时数据显示	7
5.3.4	历史记录管理	7
6	功能演示与测试结果	7
6.1	测试环境	7
6.2	测试项目与结果	7
6.3	演示效果	8

目录	2
7 个人贡献	8
7.1 蓝牙通信与上位机开发	8
7.2 用户交互与界面逻辑	8
7.3 系统任务调度与整合	8
7.4 硬件调试与测试	8
8 开发中遇到的问题与解决方案	9
8.1 蓝牙通信类问题	9
8.2 UI 交互类问题	9
8.3 系统整合类问题	9
9 项目评估与个人反思	9
9.1 项目成果评估	9
9.2 个人技术收获	10
9.3 个人反思	10
10 未来改进方向	10

1 项目概述

1.1 项目背景

随着物联网与可穿戴技术的快速发展,运动健康监测已成为嵌入式系统的重要应用方向。传统的专用运动手表采用闭源方案,开发门槛高、扩展性差,不适合教学与原型验证场景。本项目基于 STM32F103C8T6 通用微控制器平台,在开发板上设计并实现了一款低功耗运动监测设备功能原型。系统集成了 MPU6050 六轴惯性测量单元、OLED 显示面板及 BLE 低功耗蓝牙模块,并移植了 FreeRTOS 嵌入式实时操作系统,完成了运动数据的本地采集、可视化呈现与无线同步传输。项目涵盖硬件电路设计、嵌入式固件开发、通信协议设计及 PC 端上位机软件开发,形成了一个完整的嵌入式系统开发闭环。

1.2 核心功能

设备提供两大核心功能模块:

- 本地交互与显示:** 支持时间/运动双页面切换,通过旋转编码器完成菜单导航与参数设置;OLED 屏幕清晰展示日期时间、环境温度、步数、运动距离及能量消耗。
- 无线数据同步:** 运动数据以 2 秒为周期经蓝牙通道上报至 PC 端上位机软件,上位机可实时呈现各项运动指标,并具备历史数据存储与统计清零功能。

1.3 项目意义

本项目的意义在于:一方面探索了基于通用 MCU 平台构建运动监测设备的可行技术路径,验证了 FreeRTOS 在资源受限嵌入式系统中的任务调度能力;另一方面形成了一套从硬件设计到上位机开发的完整实践案例,对嵌入式系统课程具有参考价值。

2 设计目标与预期成果

2.1 设计目标

本项目从三个维度设定设计目标:

- 硬件层面:** 完成 STM32F103C8T6 最小系统电路设计,通过 I2C 总线挂载 OLED 显示屏与 MPU6050 传感器,集成 LDO 稳压电源及 Type-C 接口锂电池充电管理电路,确保各模块稳定供电与可靠通信。
- 软件层面:** 基于 FreeRTOS 合理划分 UI 显示、时间管理、传感器采集、蓝牙通信等独立任务,利用信号量与消息队列实现任务间安全数据交互,保障系统实时响应与长期稳定运行。
- 功能层面:** 扩展旋转编码器实现多级菜单导航,引入蓝牙模块与 PC 上位机进行数据同步,实现计步算法与运动数据记录功能,形成完整的用户交互闭环。

2.2 预期成果

预期交付物包括:

- 可运行的硬件开发板样机一套;
- 完整的嵌入式固件源码(含 FreeRTOS 任务框架);
- PC 端蓝牙上位机程序及源码;

- 详细设计文档与技术报告。

系统预期达到以下性能指标：OLED 界面切换流畅无卡顿，传感器数据读取准确，计步功能在模拟步行测试中准确率良好，蓝牙数据传输稳定可靠。

3 系统硬件设计

3.1 系统总体架构

本运动监测设备采用主控核心加外围模块的分层式结构。主控芯片选用 STM32F103C8T6 (ARM Cortex-M3 内核, 最高 72MHz 工作频率, 内置 64KB Flash 与 20KB SRAM), 通过 I2C、USART、GPIO 等接口分别连接 OLED 显示屏、MPU6050 六轴传感器、BLE 蓝牙模块、EC11 旋转编码器及电源管理电路。

整体架构划分为四个功能层：

- 电源层：**3.7V 锂聚合物电池经 LDO 稳压至 3.3V，为各模块提供稳定供电，同时集成 TP4056 充电管理芯片，支持通过 Type-C 接口为锂电池充电；
- 控制层：**STM32F103C8T6 作为运算核心，运行 FreeRTOS 实时操作系统，负责任务调度与数据处理；
- 感知与交互层：**MPU6050 负责运动数据采集，EC11 编码器接收用户操作指令；
- 通信与显示层：**OLED 屏幕呈现界面，蓝牙模块完成与外部设备的数据交换。

3.2 核心器件选型

关键器件的选型依据如下：

表 1: 核心器件选型表

模块	型号	关键参数	选型依据
主控 MCU	STM32F103C8T6	72MHz, 64KB Flash, 20KB RAM	性能充裕、生态完善、性价比高
显示屏	SH1106 (1.3 寸 OLED)	128×64, I2C 接口	自发光、功耗低、接线简洁
运动传感器	MPU6050	6 轴, I2C 接口	集成度高、内置 DMP
蓝牙模块	BLE 透传模块	UART, 9600bps	低功耗、兼容性好
旋转编码器	EC11	带按键	操作直观、调试方便
充电管理	TP4056	恒流/恒压充电	电路简单、成本低
锂电池	602030	3.7V, 300mAh	轻薄、容量适中

3.3 硬件引脚分配

各模块的引脚连接关系如下表所示：

表 2: 硬件引脚分配表

模块	通信接口	引脚分配
OLED	I2C1	PB6(SCL), PB7(SDA)
MPU6050	I2C1	PB6(SCL), PB7(SDA)
蓝牙模块	USART1	PA9(TX), PA10(RX)
编码器 A 相	GPIO	PA0
编码器 B 相	GPIO	PA1
编码器按键	GPIO	PA2

3.4 电源管理设计

电源管理电路由两部分组成：一是 TP4056 构成的锂电池充电管理电路，支持 5V Type-C 输入，充电电流可调，具备恒流/恒压充电及充电状态指示功能；二是低压差线性稳压器（LDO），将锂电池电压（3.7V-4.2V）稳压至 3.3V，为 MCU 及各外设提供低纹波供电。该方案电路简单、成本低廉，适合便携式原型设备的开发。

4 系统软件设计

4.1 软件整体架构

软件采用分层模块化设计，应用层代码组织如下：

Listing 1: 项目源码目录结构

```
1 Core/
2   Inc/                                # 头文件目录
3       oled.h                          # OLED 驱动
4       mpu6050.h                      # MPU6050 传感器驱动
5       pedometer.h                   # 计步算法接口
6       ui.h                          # 用户界面管理
7       encoder.h                     # EC11 编码器驱动
8       bt.h                          # 蓝牙通信模块
9       cn_font.h                     # 中文字库
10      rtc.h                          # 实时时钟
11      i2c.h                          # I2C 总线驱动
12      usart.h                        # 串口驱动
13   Src/                                # 源文件目录
14       main.c                        # 系统入口
15       freertos.c                   # FreeRTOS 任务
16       oled.c                       # OLED 驱动实现
17       mpu6050.c                    # MPU6050 读写
18       pedometer.c                  # 计步算法
19       ui.c                          # 界面与事件处理
20       encoder.c                    # 编码器轮询
21       bt.c                          # 蓝牙数据收发
22       i2c.c                        # I2C 初始化
23       usart.c                      # USART 配置
24       rtc.c                        # RTC 驱动
25       stm32f1xx_it.c               # 中断服务
```

4.2 FreeRTOS 任务设计

系统创建了两个任务，任务分工明确：

表 3: FreeRTOS 任务配置表

任务名称	功能描述	优先级	执行周期
StartDefaultTask Task_Main	初始化各硬件模块及软件状态，完成后自删除 主循环：输入采集 → 逻辑处理 → 输出更新	较高 正常	一次性 10ms

采用 10ms 固定周期的原因在于：传感器数据采集需要足够的采样率以保证计步准确性；同时 OLED 刷新频率不宜过高以免占用过多总线带宽；10ms 的周期在两者之间取得了平衡。

4.3 系统主流程

系统上电后先执行硬件外设初始化，随后启动 FreeRTOS 内核并创建任务。启动任务完成编码器、UI 状态机、计步器的初始化后自动销毁，主任务随后以 10ms 周期持续运行。

Listing 2: 主任务核心循环

```

1 void Task_Main(void *argument)
2 {
3     OLED_Init();
4     MPU6050_Init();
5     BT_Init();
6     while(1)
7     {
8         Encoder_Poll();           // 轮询编码器旋转与按键
9         UI_HandleEvent();         // 根据编码器输入切换页面/修改设置
10        MPU6050_Read();           // 读取加速度与角速度
11        Pedometer_Process();       // 执行计步状态机
12        Pedometer_GetActivity();   // 获取运动状态
13        HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
14        HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);
15        UI_Render();               // 绘制当前界面到显存缓冲区
16        OLED_Refresh();            // 批量刷新至屏幕
17        BT_Upload();               // 上传运动数据（每2秒一次）
18        BT_Process();              // 处理下行指令
19        osDelay(10);
20    }
21 }
```

4.4 UI 状态机设计

UI 模块采用有限状态机架构，共定义了 6 个状态：

表 4: UI 状态定义表

状态枚举	界面名称	功能描述
PAGE_TIME	时间主界面	显示日期、大字体时间、温度
PAGE_ACTIVITY	运动数据界面	显示步数、距离、卡路里
MENU_TIME	时间设置菜单	选择待修改的时间项
MENU_ACT	运动参数菜单	设置目标步数等
EDIT_TIME	时间编辑界面	逐项修改时分日月年
EDIT_NUM	数字编辑界面	通用数字输入编辑

状态切换逻辑清晰：在时间主界面旋转编码器可进入时间设置菜单，按键则切换至运动界面；在运动界面按键可重置计步数据；在菜单中旋转选择选项，按键进入具体编辑状态；编辑状态下旋转修改数值，按键确认并返回。

4.5 计步算法实现

计步算法是整个系统的核心功能模块之一。

4.5.1 算法原理

人体运动时，加速度数据呈现周期性变化规律。通过分析加速度矢量的幅值变化可识别步态事件：

1. **合加速度计算**: 从 MPU6050 读取三轴加速度数据, 计算合加速度幅值:

$$\text{mag} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

2. **双边沿滞回比较**: 设定高阈值 (1.2g) 和低阈值 (0.8g), 当合加速度上升穿越高阈值时判为一步的开始, 下降穿越低阈值时判为一步的结束。双阈值设计可有效避免信号在单阈值附近振荡导致的重复计数。
3. **时间滤波**: 设定 180ms 的最小步态间隔, 防止高频抖动被误判为步数。
4. **超时复位**: 当超过 2 秒无有效步态事件时, 状态机自动复位, 防止静止状态下的累积误差。

4.5.2 运动状态识别

根据步频将运动状态划分为三种模式:

表 5: 运动状态划分标准

运动状态	判定条件
IDLE (静止)	步频 < 20 步/分钟
WALKING (步行)	$20 \leq \text{步频} < 100$ 步/分钟
RUNNING (跑步)	步频 ≥ 100 步/分钟

4.6 OLED 显示驱动

OLED 驱动采用内存双缓冲机制, 开辟了 1KB 的显存缓冲区 (128×64 像素, 每像素 1bit)。所有绘图操作均在缓冲区中进行, 最后通过 OLED_Refresh 函数一次性批量传输至屏幕。这种设计避免了直接 I2C 操作导致的屏幕闪烁, 同时提升了绘制效率。显示功能支持 6×8/8×16 点阵 ASCII 字符、16×16 点阵中文字符、32×64 超大数字及基本图形绘制。

4.7 蓝牙通信模块设计

蓝牙通信是本项目的重点模块之一。

4.7.1 硬件配置

蓝牙模块通过 USART1 与主控连接, 波特率设置为 9600bps。上电初始化时通过 AT 指令完成模块参数配置:

Listing 3: 蓝牙 AT 指令配置

```

1 AT+NAME=SportsWatch // 设置设备名称
2 AT+BAUD=9600         // 设置通信波特率
3 AT+ROLE=0            // 设置为从机模式

```

4.7.2 数据协议设计

上行数据 (设备 → PC, 每 2 秒):

步数, 距离 (km), 卡路里 (kcal), 时: 分

示例：128,0.08,4.2,14:35
下行指令 (PC→ 设备):

表 6: 下行指令表

指令	功能
RESET	清空计步数据（步数、距离、卡路里归零）

4.8 资源占用分析

表 7: 资源占用统计

资源	占用	总量	利用率
Flash	~31.5KB	64KB	49.2%
RAM	~14.9KB	20KB	74.5%
主循环周期	10ms	—	—

Flash 和 RAM 均留有足够余量，便于后续功能扩展。

5 PC 端上位机开发

5.1 开发环境与工具

上位机采用 Python 语言开发，主要依赖以下库：

- **bleak**: 跨平台 BLE 通信库，负责设备扫描、连接与数据接收；
- **PyQt5**: 图形界面框架，构建主窗口及各类 UI 控件；
- **matplotlib**: 数据可视化库，用于绘制运动趋势图；
- **pandas**: 数据处理库，辅助历史记录的存储与读取。

5.2 软件架构

上位机软件采用 MVC 架构设计：

- **Model 层**: 负责数据存储与管理，包括实时数据缓存和历史记录的读写；
- **View 层**: 负责界面展示，包含数据显示面板、趋势图表和控制按钮；
- **Controller 层**: 负责业务逻辑，包括 BLE 连接管理、数据解析、界面更新触发等。

5.3 核心功能实现

5.3.1 BLE 连接管理

使用 **bleak** 库实现 BLE 设备的扫描与连接。用户点击“扫描”按钮后，程序自动搜索附近的 BLE 设备并列出；用户选择目标设备后发起连接请求，连接成功后状态栏显示“已连接”。连接管理模块采用异步回调机制处理数据接收，确保 UI 线程不被阻塞。

5.3.2 数据接收与解析

数据接收采用通知方式，设备每 2 秒发送一帧数据，上位机在回调函数中接收并解析：

Listing 4: 数据解析伪代码

```
1 async def handle_data(sender, data):
2     raw_string = data.decode('utf-8').strip()
3     parts = raw_string.split(',')
4     step_count = int(parts[0])
5     distance = float(parts[1])
6     calorie = float(parts[2])
7     time_str = parts[3]
8     # 更新UI显示与趋势图
```

5.3.3 实时数据显示

主界面布局如下：

- 左侧为数据显示面板：以数字仪表样式展示步数、距离、卡路里和温度；
- 右侧为趋势图表：以折线图展示最近 60 个数据点的步数变化趋势；
- 底部为控制栏：包含连接/断开、清零、导出历史数据等按钮。

5.3.4 历史记录管理

每次接收到的数据自动追加到本地 CSV 文件中。历史记录支持按时间排序存储、一键导出为 Excel 格式、清空全部历史数据。

6 功能演示与测试结果

6.1 测试环境

测试在普通室内环境下进行，开发板通过 USB 供电或锂电池供电运行，PC 端上位机运行于 Windows 11 系统，通过内置蓝牙适配器与设备通信。计步功能测试采用手持开发板模拟步行摆动的方式验证。

6.2 测试项目与结果

表 8: 功能测试结果表

测试项目	测试方法	结果
OLED 显示	目测各页面内容正确性	通过
编码器交互	旋转/按键测试各状态切换	通过
时间保持	断电重启后时间准确性	通过
计步准确性	模拟步行 100 步，对比计数值	误差 ≤ 3 步
蓝牙连接	10 米范围内连接稳定性	通过
数据上报	观察上位机数据更新频率	每 2 秒一次，稳定
清零功能	发送 RESET 指令	数据归零，通过
历史记录	检查 CSV 文件存储内容	数据完整

6.3 演示效果

开发板 OLED 屏幕可清晰显示时间页面（日期、大字体时分、环境温度）和运动页面（步数、距离、卡路里）。PC 上位机通过 BLE 接收数据，实时更新各项运动指标。经实际测试验证，系统运行稳定，界面切换流畅，蓝牙数据传输可靠，各项功能均达预期。

7 个人贡献

本人完成的主要工作如下：

7.1 蓝牙通信与上位机开发

独立完成了蓝牙通信模块的全部设计与实现：

- 设计并实现了蓝牙模块的 AT 指令配置流程（设备命名、波特率设置、角色配置）；
- 制定了上行数据格式规范（步数、距离、卡路里、时间的 CSV 格式编码）；
- 实现下行指令解析机制（RESET 清零指令的接收与响应）；
- 独立开发了 PC 端 Python 上位机程序，包括 BLE 设备扫描与连接管理、异步数据接收与解析、实时数据显示面板、运动趋势折线图、历史记录 CSV 存储与导出功能。

7.2 用户交互与界面逻辑

负责 UI 模块的设计与实现：

- 设计了 6 状态 UI 状态机的整体框架与状态切换逻辑；
- 实现了时间主界面（含大字体数字显示）与运动数据界面的渲染函数；
- 完成了编码器事件（旋转方向、短按、长按）与 UI 状态切换的映射逻辑；
- 实现了时间设置、运动参数设置等菜单导航与数值编辑功能。

7.3 系统任务调度与整合

基于 FreeRTOS 完成了系统级整合：

- 设计了主任务 10ms 周期的任务循环架构，合理排布各模块的执行时序；
- 协调了 UI 渲染、传感器读取、蓝牙通信等模块之间的资源竞争问题；
- 负责全部代码的工程整合与版本管理，确保各模块接口对齐。

7.4 硬件调试与测试

参与硬件层面的工作：

- 参与模块布局规划，确定了核心器件的安装位置方案；
- 完成了 OLED、MPU6050、蓝牙模块的 I2C 和 UART 总线接线；
- 协助排查了 I2C 通信时序问题；
- 编制了功能测试用例，执行了计步准确性、蓝牙连接稳定性等测试。

个人承担硬件物料费用约 25 元。

8 开发中遇到的问题与解决方案

8.1 蓝牙通信类问题

表 9: 蓝牙相关问题及解决

问题描述	解决方案
蓝牙模块 AT 指令无响应 数据上报偶尔出现乱码 上位机连接不稳定	检查发现 TX/RX 接线反接，交换后恢复正常 串口波特率误设为 115200，统一配置为 9600 后正常 增加连接超时重试机制，优化异常处理

8.2 UI 交互类问题

表 10: UI 相关问题及解决

问题描述	解决方案
编码器旋转时有漏检 菜单导航逻辑混乱 大字体显示出现残影	优化轮询采样逻辑，增加消抖延时 重新梳理状态转换图，明确定义合法转移路径 刷新前执行清屏操作

8.3 系统整合类问题

表 11: 系统整合相关问题及解决

问题描述	解决方案
模块初始化顺序不当 I2C 总线冲突 RAM 占用接近上限	按硬件依赖关系调整顺序 分时复用，通过互斥量保证独占访问 将只读数据移至 Flash (const 修饰)

9 项目评估与个人反思

9.1 项目成果评估

成功之处：

- 全链路打通：从硬件搭建、嵌入式固件开发到 PC 上位机实现，形成了完整的工作流；
- 编码器交互逻辑清晰，操作体验良好；
- 蓝牙通信稳定可靠，上位机功能完整，具备实际可用性；
- 资源利用率合理，Flash 和 RAM 均留有扩展余量。

不足之处：

- 计步算法对不同运动模式的适应性有待提升；
- 设备未实现低功耗休眠模式，电池续航约 4-6 小时；
- 运动数据仅在 RAM 中保存，掉电后丢失；
- 当前为开发板形态，尚未集成至紧凑的穿戴式外壳中。

9.2 个人技术收获

通过本次项目，我获得了以下成长：

1. **嵌入式 RTOS 实战能力**：深入理解了 FreeRTOS 的任务调度机制、任务间同步与通信方法，掌握了在资源受限系统中合理划分任务的工程方法。
2. **蓝牙协议栈应用能力**：熟悉了 BLE 串口透传模块的 AT 指令配置流程，掌握了 BLE 通信的基本原理与数据收发流程。
3. **Python 上位机开发能力**：熟练运用了 `bleak` 库进行 BLE 通信开发，深化了 PyQt5 图形界面编程技能。
4. **系统集成与调试能力**：锻炼了软硬件联调的工程能力，培养了模块化开发的思维方式。

9.3 个人反思

本次项目让我深刻认识到，嵌入式系统开发不仅仅是编写 MCU 代码，更需要从系统层面统筹考虑硬件限制、实时性要求和用户体验等多维因素。在蓝牙调试阶段，一根接反的线缆可能导致数小时的排查，这提醒我在硬件连接环节必须保持严谨细致的工作习惯。

10 未来改进方向

针对本项目的不足，后续改进方向如下：

1. **计步算法优化**：引入自适应阈值机制，根据最近若干步的加速度幅值动态调整高低阈值，提升不同运动模式下的适应性。
2. **低功耗设计**：增加 FreeRTOS 的 Tickless 空闲模式，配置 MPU6050 的低功耗模式，增加电量监测功能。
3. **数据持久化**：外扩 SPI Flash 芯片，实现运动数据的本地存储。
4. **双向控制协议**：完善蓝牙下行指令集，增加目标步数设置、时间同步等远程控制功能。
5. **集成化设计**：设计专用 PCB 整合各模块，减小体积，为后续可穿戴化改造奠定基础。

参考文献

- [1] STMicroelectronics. STM32F103xC/D/E Reference Manual[Z]. 2019.
- [2] 中景园电子. 132×64 点阵 OLED/PLED 驱动控制器 SH1106 数据手册 [Z]. 版本 V0.2, 2011.
- [3] InvenSense. MPU-6000/MPU-6050 Product Specification[Z]. 2013.
- [4] 中国国家标准. GB2312-80 信息交换用汉字编码字符集·基本集 [S]. 北京: 中国标准出版社, 1980.
- [5] Barry R. Mastering the FreeRTOS Real Time Kernel[Z]. 2016.